

META MINDS

SAATHI

Human-Centered Multimodal AI Response Agent

Production-Level Product Requirements & Engineering Design Document

Team	Meta Minds
Team Leader	Mohd Junaid Pasha
Team Member	Muhammad Saif Patel
Team Member	Mustyala Shiva Sai Tej
Target Event	TechFusion 2026
Document Version	1.0

“Tell us what is happening. We’ll help with what happens next.”

1. Executive Summary

SAATHI is a multimodal, agentic human-assistance platform. The user does not need to understand which application, department, hospital, service, emergency number or procedure they need. They simply communicate naturally. SAATHI determines: what the user is trying to communicate; whether there is immediate risk; what information is missing; which specialist capability should handle the request; which external service is appropriate; whether user authorization is required; what action should be taken; and what needs to be monitored afterward.

- Listen → Understand → Assess → Reason → Ask → Authorize → Act → Verify → Follow up → Learn
- SAATHI is not positioned as an AI chatbot. It is positioned as: “A personal AI response agent that turns human intent into coordinated real-world action.”

2. Product Vision

Create an AI layer through which ordinary people can access complex societal infrastructure simply by talking. The user should not have to know which government department exists, which hospital to call, which emergency number to remember, which form to fill, which route is safest, or which service is relevant.

- Human intent → appropriate action

3. Product Philosophy

SAATHI follows six principles.

- Human-first — AI assists the human and does not replace the human’s authority over consequential decisions.
- Action over conversation — the objective is to help the user accomplish the correct next step.
- Permission before consequential action — explicitly request authorization before contacting third parties, sharing sensitive information, sharing location, creating official complaints, making appointments or initiating payments unless a separately governed emergency protocol applies.
- Safety before intelligence — when uncertain, the system should become more conservative.
- Explainability — for consequential decisions, explain what was understood, what was detected, what information was used, why an action is recommended and what uncertainty remains.
- Privacy by design — collect the minimum necessary data.

4. The Core Product

SAATHI consists of seven layers.

- Human → Multimodal Interface → Context Engine → Intent + Risk Engine → Agent Orchestrator → Action Engine → Follow-up Engine / Outcome Memory

5. Target Users

Primary users include individual citizens, elderly people, people with accessibility needs, students/young adults, and authorized family caregivers.

6. Specialist Agents

SAATHI should not be one giant prompt. Use specialist agents.

- Medical Coordination Agent
- Disaster Response Agent
- Civic Agent
- Accessibility Agent
- Safety Agent
- Family Assistance Agent

- Lost Person / Reunification Agent

7. Medical Coordination Agent

Purpose: emergency triage support; symptom information gathering; hospital discovery; appointment assistance; authorized communication; follow-up. Important boundary: it must not claim to diagnose disease.

- Example: “Chest heaviness can have many causes, including some that require urgent evaluation. I cannot diagnose the cause. Based on what you’ve told me, urgent medical evaluation may be appropriate.”

8. Disaster Response Agent

Capabilities: weather risk, flood risk, heat risk, severe weather alerts, supported hazards, safe-route discovery, shelter discovery and emergency preparation.

- Weather Data + Geospatial Data + Historical Data + ML Risk Model → Local Risk Estimation → Personalized Response

9. Civic Agent

Capabilities: understand complaint; classify department; identify required information; generate complaint; submit through authorized channel; store reference number; track status; remind user; escalate when appropriate.

- Example: “Garbage has not been collected for five days.” → category, location, severity, department and required evidence.

10. Accessibility Agent

Capabilities: OCR, document understanding, scene description, object identification, text-to-speech, speech-to-text, multilingual communication and simplified explanations.

- Example: “What does this document say?” → camera → document understanding → spoken explanation.

11. Safety Agent

Capabilities: suspicious-message detection, scam detection, phishing detection, suspicious-link analysis, social engineering detection and emergency safety workflow.

- Example: “Someone is asking me to transfer money immediately.” → identify urgency, financial request and pressure to avoid verification.

12. Family Assistance Agent

Optional long-term capability. With explicit consent: elderly check-ins, medication reminders, scheduled welfare checks, emergency escalation and trusted-contact notification. Never silently monitor people.

13. Lost Person / Reunification Agent

Long-term capability. Inputs can include authorized report, approximate age, physical description, location, time, clothing and photographs where legally appropriate. The system produces a potential match, not a confirmed identity. All identity decisions require authorized human verification.

14. Core Voice Experience

The voice interface is the heart of SAATHI. Use real-time speech-to-speech rather than a slow speech → transcription → LLM → TTS pipeline for the main conversational experience.

- Target experience: natural conversation with interruption, barge-in, silence, clarification, multilingual speech, code switching, confirmation and tool execution.

15. Voice Language Strategy

Initial hackathon languages: English, Hindi and Telugu. Architecture must support expansion. Language should be detected automatically and the assistant should respond in the user's preferred language.

16. Voice Emotion / Tone

Voice characteristics may be used as contextual signals such as unusually distressed speech, crying, shouting, panic-like speech, confusion or urgency. Never claim that voice emotion diagnoses a medical condition. Voice affect is a supporting feature, not a medical diagnostic model.

17. Emergency Mode

Normal conversation uses understanding → answer. Potential emergency uses risk detection → emergency mode → minimal questions → confirm location → offer/initiate appropriate help → follow up.

18. Emergency Decision Policy

Never let an LLM directly decide that a person has a disease. Create a deterministic safety policy layer.

Example: if a potentially life-threatening symptom pattern is present and the user indicates current symptoms, set high risk, recommend urgent evaluation, provide an emergency option, ask location if necessary and avoid diagnosis or dangerous reassurance.

19. Action Authorization

Every tool must declare risk level, whether confirmation is required, required permissions, data shared, external party, reversibility and audit requirement.

- Example action: `contact_hospital`; `risk_level=high`; `requires_confirmation=true`; `data_shared=[name, symptoms, location]`; `reversible=false`; `audit_required=true`.

20. Agent Architecture

Use an orchestrator. The orchestrator supports agent routing, tool calls, context propagation, approval gates, retries, timeouts and fallbacks. Specialist agents include Medical, Disaster and Civic agents.

21. Recommended Technology Stack

Frontend: React Native + Expo for the mobile application, with native Kotlin/Swift modules where performance-critical. Web dashboard: Next.js + TypeScript. Backend: TypeScript + NestJS for the primary API and Python + FastAPI for AI/ML services.

22. AI Stack

Use the strongest currently available model appropriate to the workload. Use a GPT-5.6-class reasoning model for complex orchestration and a cost-sensitive GPT-5.6-class model for high-volume tasks. Use the Responses API / agentic platform for structured, tool-based workflows.

23. Voice

Primary: OpenAI Realtime API. Transport: WebRTC. Fallback: WebSocket. Phone/SIP integration: SIP-compatible telephony provider.

24. ML Stack

Python with PyTorch, scikit-learn, XGBoost, pandas and NumPy. Serving with FastAPI and ONNX Runtime where appropriate. Experiments with MLflow. Evaluation with custom benchmark suites, model regression tests and safety evaluation datasets.

25. Database

Primary database: PostgreSQL. Use it for users, profiles, conversations, cases, actions, consent, audit events, service providers, appointments and other transactional data.

26. Vector Database

Use pgvector initially. Keep vector search inside PostgreSQL to reduce moving parts, while retaining transactional consistency. Introduce a separate vector database only if scale requires it.

27. Cache

Redis for sessions, rate limiting, temporary state, realtime presence, job coordination and short-lived agent context.

28. Event Bus

Production: Kafka. Hackathon: Redis Streams. Long-term architecture: domain event → event bus → agents / analytics / notifications / audit.

29. Workflow Engine

For production-grade long-running workflows use Temporal. Example: hospital contacted → wait → check status → if unresolved, escalate. Do not make an LLM responsible for timers or durable workflow state.

30. Search

Use OpenSearch for incident search, audit search, service search, logs and operational queries.

31. Maps / Routing

Use Google Maps Platform for Maps, Places, Routes API and route matrix, while abstracting the provider behind a RoutingProvider interface so Mapbox, OpenStreetMap/OSRM, HERE or government GIS can be added later.

32. Cloud

Production recommendation: AWS with EKS, RDS PostgreSQL, ElastiCache Redis, S3, CloudFront, WAF, KMS, Secrets Manager and CloudWatch. Google Cloud is a valid alternative. For the 24-hour hackathon, do not deploy Kubernetes unless the team is already excellent at it; use Docker plus managed container deployment.

33. Authentication

Use Auth0, Clerk or Firebase Auth for the prototype. Enterprise alternative: OIDC + Keycloak. Support phone OTP, email, passkeys, OAuth and device binding. Never store raw passwords.

34. Security

Implement TLS everywhere, encryption at rest, KMS, secret manager, short-lived tokens, refresh-token rotation, RBAC, ABAC for sensitive actions, audit logs, rate limiting, WAF, input/output validation, prompt-injection detection, tool allowlists, SSRF protection, malware scanning and signed webhooks.

35. Data Classification

Classify information as PUBLIC, INTERNAL, CONFIDENTIAL, SENSITIVE and HIGHLY SENSITIVE. Health, precise location and government identifiers are sensitive; medical information and emergency events are highly sensitive.

36. Privacy Architecture

Follow data minimization, purpose limitation, consent, retention limits, access control, deletion and auditability. Build privacy into the prototype from day one rather than adding it later.

37. Consent Model

Every sensitive operation should have a consent record containing user_id, purpose, data_types, recipient, timestamp, expiry, version and revoked_at.

38. Agent Tool Registry

Every tool should be typed. Examples: find_nearby_hospitals(), get_weather_risk(), calculate_safe_route(), create_civic_complaint(), contact_service(), send_notification(), share_location(), schedule_followup(), get_case_status(). Tools must never be arbitrary shell commands.

39. Tool Permission System

Risk classes: LOW for public weather; MEDIUM for hospital search; HIGH for location sharing; CRITICAL for emergency contact. Every tool declares its risk and approval requirements.

40. Agent Guardrails

Before tool execution: user intent → policy check → permission check → data minimization → action preview → user approval → tool execution.

41. Human-in-the-Loop

For consequential actions, explicitly request confirmation. For emergencies, use a separately governed emergency protocol rather than allowing an LLM to bypass approval arbitrarily.

42. AI Safety Layer

Build a dedicated Safety Gateway checking prompt injection, unsafe medical advice, privacy leakage, unauthorized tool calls, dangerous instructions, hallucinated contacts, unsupported claims and suspicious external content.

43. Structured AI Outputs

Never parse free-form LLM text to decide whether to make a critical action. Use JSON Schema, validate server-side, and include fields such as intent, risk_level, confidence, needs_clarification, recommended_action and requires_confirmation.

44. Emergency Risk Engine

Use a hybrid architecture: rules + ML classifier + LLM contextual reasoning + safety policy. Never use only an LLM.

45. Disaster ML

For the hackathon, implement a realistic prototype using historical/synthetic data. Features can include rainfall, temperature, elevation, drainage, historical incidents, road characteristics, population and time. Output risk_score, risk_category, confidence, prediction_horizon and affected_zone.

46. Civic NLP

Pipeline: Voice → Transcription → Language Detection → Intent Classification → Entity Extraction → Department Classification → Required Information → Complaint Draft.

47. Accessibility Vision Pipeline

Pipeline: Camera → image preprocessing → OCR / vision model → scene understanding → spatial reasoning → natural language → voice response.

48. Case Management

Every real-world issue becomes a Case. A case contains case ID, type, status, risk, creation time, actions, next step and resolution state.

49. Follow-Up Engine

SAATHI should continue after the first action: ask whether the hospital contacted the user, whether an ambulance arrived, whether a complaint received a reference number, or whether the user is safe. This converts an assistant into a response system.

50. Notification Layer

Support push, SMS, email, WhatsApp where officially integrated, voice call and in-app notifications. Use a NotificationProvider abstraction so providers can change without rewriting business logic.

51. Telephony

Production: SIP, a compliant telephony provider and carrier integration where available. Hackathon: simulated hospital call or controlled test number. Never repeatedly call real emergency services for a demonstration.

52. Hospital Portal

Create a separate hospital web application showing incoming case, user-provided name, reported issue, authorized location, time, risk and actions such as accept, request more information or decline.

53. Emergency Operations Dashboard

Show active cases, high/medium/low risk, map with incidents/hospitals/shelters/emergency vehicles/risk zones, and a case timeline.

54. AI Trace Viewer

Expose safe operational reasoning summaries, not hidden chain-of-thought. Show intent detected, risk assessment, why, action, authorization, tool and result.

55. Observability

Production stack: OpenTelemetry. Track latency, model calls, tool calls, errors, token usage, action success, agent handoffs and human approvals. Provide an AI trace from conversation to outcome.

56. Evaluation System

Create evaluation datasets for medical, disaster, civic and accessibility scenarios. Measure intent accuracy, risk classification accuracy, hallucination rate, tool selection accuracy, escalation accuracy, unsafe-action rate, latency and task completion.

57. Critical Safety Metric

Track Unsafe Action Rate. Target: 0% for production release. A wrong answer is bad; an unauthorized dangerous action is unacceptable.

58. Latency Targets

Voice first response target: under 800 ms. Normal agent target: under 3 seconds. Tool execution target: under 5 seconds where external APIs permit. Emergency workflows should minimize unnecessary conversational turns.

59. Reliability

Production target: 99.9% service availability. During the hackathon prioritize correctness, then reliability, then polish, then breadth.

60. Failure Modes

Handle no internet, model unavailability, hospital API unavailability, missing location, ambiguous intent and tool failure. Never pretend an action succeeded.

61. Never Hallucinate Actions

Never say “I contacted the hospital” unless the tool returned success. Say “The contact request was successfully initiated” or “I couldn’t reach the hospital’s system.”

62. Never Hallucinate Services

Hospital/service information must come from verified provider databases, trusted APIs or manually curated hackathon datasets. Never let the LLM invent phone numbers, addresses, departments or government portals.

63. Hackathon MVP

Build four workflows perfectly: Medical — voice → risk → hospital → authorization → simulated contact → follow-up; Disaster — voice → location → ML risk → safe route/shelter; Civic — voice → complaint → classification → structured case → tracking; Vision/accessibility — camera → understand → explain by voice.

64. The “Magic” Demo

Open with almost nothing: a single large “Talk to SAATHI” button. The user speaks naturally. No category selection. The AI determines the correct specialist workflow.

65. Hackathon Demo Script

Scene 1: medical emergency-style conversation, risk escalation, hospital selection and case reception. Scene 2: flood concern, location, risk model and safe route. Scene 3: ignored civic complaint, classification and tracking. Scene 4: camera-based document explanation.

66. The Judge “Wow” Moment

After four workflows, reveal that they are not four applications but one human interface connected to an agent orchestration layer. Show the architecture: SAATHI → intent + risk → specialist agents → services → action → follow-up.

67. Repository Structure

Use a monorepo with apps/mobile, apps/web, apps/hospital-portal, apps/operations-dashboard; services/api, ai, risk-engine, workflow, notifications, integrations; packages/types, schemas, auth, ui, config; models; infrastructure; data; docs.

68. API Design

Core endpoints: /v1/conversations, /v1/agent/run, /v1/cases/:id, /v1/actions/:id/approve, /v1/actions/:id/reject, /v1/hospitals/nearby, /v1/disaster/risk, /v1/civic/complaints and /v1/civic/complaints/:id.

69. Database Core Tables

users, profiles, consents, conversations, messages, memories, cases, case_events, risk_assessments, agent_runs, tool_calls, actions, action_approvals, service_providers, hospitals, complaints, notifications,

locations, audit_logs, model_predictions and feedback.

70. Case State Machine

CREATED → ASSESSING → AWAITING_USER → AUTHORIZED → ACTION_PENDING → ACTION_EXECUTING → ACTION_COMPLETED → FOLLOW_UP → RESOLVED. Failure path: ACTION_FAILED → RETRY / FALLBACK / HUMAN.

71. Agent State

Persist agent_run_id, case_id, intent, risk_level, current_agent, current_state, pending_action, required_approval, tool_state, created_at and updated_at.

72. First 2 Hours

00:00–00:15 freeze scope. 00:15–00:30 create GitHub repository. 00:30–00:45 create architecture diagram. 00:45–01:00 create environment variables. 01:00–01:30 create PostgreSQL schema. 01:30–02:00 build API skeleton + authentication.

73. Hours 2-5

AI lead builds voice → orchestrator → tool. ML lead builds disaster-risk model. Backend builds case/action/consent APIs. Frontend builds voice-first UI. At hour 5, one complete end-to-end flow must work.

74. Hours 5-8

Finish medical workflow: voice → risk → hospital search → hospital selection → approval → case creation.

75. Hours 8-12

Build Disaster Agent: location → risk model → map → safe route.

76. Hours 12-15

Build Civic Agent: complaint → classification → structured case → tracking.

77. Hours 15-18

Build Vision Agent: camera → vision → explanation → voice.

78. Hours 18-20

Build Operations Dashboard: active cases, risk, map, agent activity, action history and timeline.

79. Hours 20-21

Build AI Trace Viewer showing User → Intent → Risk → Agent → Tool → Approval → Action.

80. Hours 21-22

Run failure tests, latency tests, demo rehearsal and backup scenarios.

81. Hours 22-23

Freeze features. Only fix, polish and stabilize. Do not introduce new functionality.

82. Hours 23-24

Final presentation: live demo, architecture, societal impact, technical depth, safety and future roadmap.

83. What NOT to Build During the Hackathon

Do not waste time on custom authentication infrastructure, Kubernetes, custom LLM training, complicated microservices, a native iOS app, blockchain, custom vector database, unnecessary animations, 20 agents or 50 integrations. Build one excellent agentic platform.

84. The “Production Illusion”

Real: voice, ML, agent routing, tool calls, database, consent, case management, map, dashboard, hospital simulation and follow-up. Simulated: hospital integration, government API, emergency dispatch and disaster sensors. Clearly label simulations and never pretend a simulated hospital call was actually sent.

85. Production Roadmap

Phase 1: hackathon MVP. Phase 2: pilot with one hospital, one NGO, one municipal body and one emergency workflow. Phase 3: regional deployment. Phase 4: government/health integrations. Phase 5: nation-scale platform.

86. Long-Term Expansion

Education Agent for scholarships/admissions/forms/deadlines; Employment Agent; Financial Safety Agent; Disaster Agent; Public Services Agent; Family Agent; Accessibility Agent.

87. The Real Product Moat

The moat is not “we use GPT.” The moat is action infrastructure, safety policies, consent architecture, personal context, service integrations, case management, outcome feedback, domain-specific ML, agent evaluation and trust.

88. Success Metrics

User: task completion rate, time-to-action, abandonment rate, satisfaction. AI: intent accuracy, risk accuracy, hallucination rate, tool selection accuracy. Safety: unsafe action rate, unauthorized data-sharing rate, false escalation rate, missed escalation rate. Societal: emergency response coordination time, complaints successfully resolved and users successfully assisted.

89. North-Star Metric

Successful Human Outcomes. Do not optimize for conversations, tokens or responses. Optimize for how many real-world problems SAATHI successfully helped a human resolve.

90. Brand

Recommended name: SAATHI. Meaning: companion / one who accompanies you. Full-name option: SAATHI — Situational AI Assistance & Human Response Intelligence. Taglines: “Tell us what is happening. We’ll help with what happens next.” and “One conversation. Any situation.”

91. Final Architecture

Final flow: Human → Voice/Text/Camera → Multimodal Gateway → Context + Memory → Intent Engine → Risk/Safety → Agent Orchestrator → Medical/Disaster/Civic Agents → services → Action Gate → Human Approval → Action Execution → Case Management → Follow-up Agent → Outcome/Memory.

92. The One-Sentence Definition

“SAATHI is a multimodal AI response agent that understands a person’s situation through natural conversation, assesses risk, selects specialized AI capabilities, obtains the required authorization, coordinates with real-world services, executes the next action, and follows up until the case is resolved.”

93. The Winning Pitch

“Imagine you’re alone at home and suddenly you don’t feel well. You don’t want to search Google. You don’t want to find the right hospital. You don’t want to figure out which number to call. You just want someone to understand what is happening and help. That is why we built SAATHI. You talk. SAATHI understands the situation. It determines what kind of help may be needed, asks for permission before consequential actions, coordinates the appropriate service, and stays with the case afterward. And the same interface can help when the problem isn’t medical — a flood, a civic complaint, an accessibility barrier, or a safety threat. We aren’t building another AI chatbot. We’re building an AI layer between humans and the services they need.”

94. Final Build Priority

MUST HAVE: voice conversation; multimodal input; intent routing; risk engine; agent orchestration; tool calling; human approval; medical demo; disaster demo; civic demo; case management; follow-up; audit trail. SHOULD HAVE: vision, multilingual, map, hospital dashboard, operations dashboard, ML disaster model, AI trace viewer. NICE TO HAVE: proactive alerts, family mode, scam detection, lost-person workflow, advanced personalization, real telephony. DO NOT COMPROMISE: safety, consent, authorization, data privacy, no fabricated actions, no fabricated service information and clear distinction between prototype simulation and real integration.

95. Final Technical Principle

Do not build SAATHI as an LLM with a giant prompt. Build LLM + ML + deterministic policy + tools + workflows + memory + permissions + real-world integrations + human oversight. The LLM should be the reasoning interface, not the entire system.

96. Recommended Build Order — Foundation

Before feature work, establish the monorepo, TypeScript/Python services, environment management, Docker setup, PostgreSQL schema, Redis, authentication, shared types, API contracts, logging, error handling and CI.

97. Recommended Build Order — Intelligence

Implement structured intent classification, deterministic risk policies, orchestrator routing, specialist-agent prompts, typed tools, approval gates, memory policy and evaluation harness before adding a large number of integrations.

98. Recommended Build Order — Action

Implement hospital discovery, civic case creation, disaster risk lookup, safe-route calculation, notification abstraction and case state machine. Every external action must return a verifiable success/failure result.

99. Recommended Build Order — Experience

Polish the voice-first mobile interface, map view, hospital portal, operations dashboard, AI trace viewer and case timeline. The experience should make the underlying complexity visible without exposing private model reasoning.

100. Recommended Build Order — Safety

Run adversarial tests for prompt injection, unauthorized tool calls, hallucinated phone numbers, fake success messages, incorrect escalation, missing consent, location leakage, sensitive-memory persistence and external API failures.

101. Demo Readiness Gate

Do not present until all four core flows can be executed from a clean state, every simulated integration is clearly labeled, failure paths have been rehearsed, the dashboard shows real state transitions, and the team can explain exactly which parts are ML, LLM, deterministic policy, workflow and integration.

102. Production Readiness Gate

Before any real deployment, add legal/privacy review, data-protection controls, medical safety review, threat modeling, provider verification, incident response, access review, penetration testing, model evaluation, monitoring, disaster recovery, retention/deletion controls and human escalation procedures.

103. Final Technical Principle & Team Directive

The system succeeds when a human can speak naturally about a real situation and SAATHI can safely convert that conversation into a verified next action. The team should resist feature bloat, build the platform loop first, demonstrate four deep workflows, and make every consequential action observable, authorized and auditable.

- Core loop: Human → Understand → Assess → Ask → Authorize → Act → Verify → Follow up → Resolve.
- Team directive: build one coherent response platform, not a collection of disconnected AI features.

META MINDS — TEAM & OWNERSHIP

Role	Member	Primary Ownership
Team Leader	Mohd Junaid Pasha	Product direction, architecture, orchestration, demo, integration
AI/ML	Muhammad Saif Patel	ML models, evaluation, agent intelligence, risk engine
Full-Stack / Systems	Mustyala Shiva Sai Tej	Backend, database, workflows, APIs, deployment

Build North Star

One conversation. Any situation. Safe, authorized, observable real-world action.